

Game Design Document (GDD)

Game Overview

- **Title:** ARR You Drunk Yet?
- **Genre:** 3D Physics-Based Trajectory / Arcade Puzzle
- **Target Platform:** PC (OpenGL / GLFW)
- **Core Concept:** Control a cannon to launch a drunk, ragdolling pirate into floating beer bottles to hit level quotas before landing in the ocean or running out of flight time.

Gameplay & Mechanics

Primary Objective

- Collect enough floating beer bottles in a single launch flight to meet the *Bottle Quota* of each level.

Controls

- **Left / Right Arrow Keys:** Adjust cannon launch angle
- **Up / Down Arrow Keys:** Increase / decrease launch power
- **Spacebar:** Fire cannon / launch pirate
- **R:** Restart current level (resets current run bottles, retains overall campaign score)
- **L:** Skip level (debug/developer key)
- **W / A / S / D:** Pan camera controls
- **Esc:** Exit game

Key Physics & Game Systems

1. **Flight Physics:** Integrates gravity and dynamic air drag via [melonyx::PhysicsWorld](#).
2. **Obstacles (Hanging Crates):** Dynamic pendulums linked by constraint rods that block trajectory paths.
3. **Power-Ups:**
 - **Bouncer (Yellow):** Reflects pirate upward with increased velocity.
 - **Boost Ring (Cyan):** Accelerates flight along current directional vector.

4. Win / Fail Conditions:

- **Win:** Hit or exceed the level quota and land safely. Clear Level 10 to beat the game!
- **Game Over:** Miss quota, drop into the water, go out-of-bounds, or hit the 15-second flight timer.

The “Big 3” (Three Main Systems)

1. Collisions

- a. Distance Check: The game measures how close the pirate is to a bottle or obstacle. If the distance is smaller than their combined sizes, a collision happens.

```
float distToCrate = glm::length(pirate.Position - gCrate.Position);
float distToMast = glm::length(pirate.Position - gMastAnchor.Position);
if (distToCrate < pirate.radius + gCrate.radius ||
    distToMast < pirate.radius + gMastAnchor.radius) {
    pirate.onCollisionHit(glm::length(pirate.Velocity));
}
```

- b. Bouncing Apart: When the pirate hits a hard obstacle, the physics engine flips his velocity to make him bounce off.

```
if (gState == GameState::LAUNCHED) {
    pWorld.Update(timestep_sec); // velocity handled in particlecontact.cpp
    pirate.updateRotation(timestep_sec);
    gFlightElapsed += timestep_sec;
}
```

When `pWorld.Update()` is called, the physics engine automatically:

- Detects collisions and rod constraint limits
- Creates `ParticleContact` objects
- Calls `resolveVelocity()` under the hood
- Updates the pirate's Position and Velocity

2. Rods & Cables

- If the crate swings too far away or pushes too close, the rod pulls or pushes it back to keep the exact length

```
if (currLen == length)
{
    return nullptr;
}

ParticleContact* ret = new ParticleContact();
ret->particles[0] = particles[0];
ret->particles[1] = particles[1];

glm::vec3 dir = particles[1]->Position - particles[0]->Position;
dir = glm::normalize(dir);

if (currLen > length)
{
    ret->contactNormal = dir;
    ret->depth = currLen - length;
}
else
{
}
```

3. Force Generators

- a. **Gravity:** Constantly pulls the pirate straight down ($F = m * g$)
- b. **Air Drag:** Pushes back against the pirate based on how fast he flies (slowing him down in the air)

```
void WindForceGenerator::UpdateForce(Particle* particle, float duration) {
    if (!particle) return;

    // Multiply by mass so windAcceleration behaves like GRAVITY does --
    // an acceleration value, not a raw force -- keeping units consistent
    // for level tuning (heavier particles aren't blown around less).
    particle->AddForce(windAcceleration * particle->mass);
}
```

```
GravityForceGenerator    gravityGen(glm::vec3(0.0f, -GRAVITY, 0.0f));
melonyx::DragForceGenerator dragGen(DRAG_K1, DRAG_K2);
melonyx::WindForceGenerator windGen(glm::vec3(0.0f, 0.0f, 0.0f));
```

Core Implemented Mechanics & Code

1. Aiming & Launch System

- **Mechanic:** Controls the cannon's pitch angle and launch velocity, showing a dotted trajectory preview before firing.
- **Controls:**
 - **Left / Right** Arrow Keys: Pitch rotation
 - **Up / Down** Arrow Keys: Launch power
 - **Spacebar**: Fire cannon

```
//  
// Dynamic Trajectory Generation for Preview  
//  
vector<glm::vec2> simulateArc(float angleDeg, float power, float waterLine) {  
    vector<glm::vec2> pts;  
    glm::vec2 pos = gCannon.muzzlePosition();  
  
    float rad = glm::radians(angleDeg);  
    glm::vec2 vel(cosf(rad) * power, sinf(rad) * power);  
  
    for (int i = 0; i < 500; i++) {  
        vel.y -= GRAVITY * PREVIEW_DT;  
        vel *= (1.0f - DRAG_K1 * PREVIEW_DT);  
        pos += vel * PREVIEW_DT;  
        pts.push_back(pos);  
        if (pos.y < waterLine) break;  
    }  
    return pts;  
}
```

```
void launchPirate() {  
    pWorld = melonyx::PhysicsWorld();  
    pirate.reset(gCannon.muzzlePosition());  
    pirate.Velocity = gCannon.fire();  
    pirate.onLaunch(gCannon.power);  
  
    pWorld.AddParticle(&pirate);  
    pWorld.forceRegistry.Add(&pirate, &gravityGen);  
    pWorld.forceRegistry.Add(&pirate, &dragGen);  
  
    pWorld.Particles.push_back(&gMastAnchor);  
    pWorld.AddParticle(&gCrate);  
    pWorld.Links.push_back(&gCrateRod);  
  
    gFlightElapsed = 0.0f;  
    gState = GameState::LAUNCHED;  
}
```

2. Flight Physics (Gravity + Air Drag)

- **Mechanic:** Uses the `melonyx` physics world to apply real-time linear gravity and dynamic air resistance to the flying pirate.

```
GravityForceGenerator gravityGen(glm::vec3(0.0f, -GRAVITY, 0.0f));  
DragForceGenerator    dragGen(DRAG_K1, DRAG_K2);
```

```
if (gState == GameState::LAUNCHED) {  
    pWorld.Update(timestep_sec);  
    pirate.updateRotation(timestep_sec);  
    gFlightElapsed += timestep_sec;
```

3. Dynamic Swinging Obstacle (Pendulum Constraint)

- **Mechanic:** A hanging crate swinging on a rigid distance rod connected to a moving anchor mast, creating dynamic level obstacles.

```
void updateMovingObstacles(float appTime, int level) {  
    if (level == 1) return;  
  
    if (level % 3 == 0) {  
        gMastAnchor.Position.y = gBaseAnchorPos.y + sinf(appTime * 2.5f) * 40.0f;  
        gMastAnchor.Position.x = gBaseAnchorPos.x + cosf(appTime * 1.2f) * 30.0f;  
    }  
    else if (level % 2 == 0) {  
        gMastAnchor.Position.y = gBaseAnchorPos.y + sinf(appTime * 2.0f) * 35.0f;  
    }  
    else {  
        gMastAnchor.Position.x = gBaseAnchorPos.x + cosf(appTime * 1.8f) * 60.0f;  
    }  
}
```

4. Interactive Power-Ups

- **Mechanic:** Contact triggers modifying flight vectors:
 - **Bouncer (Yellow):** Reflects pirate upward with speed boost.
 - **Boost Ring (Cyan):** Accelerates flight along current trajectory direction.

```
void checkPowerUpCollisions(pirategame::Pirate& p) {  
    for (auto& pu : gPowerUps) {  
        if (pu.collected) continue;  
  
        float dist = glm::length(p.Position - pu.position);  
        if (dist < (p.radius + pu.radius)) {  
            pu.collected = true;  
  
            if (pu.type == PowerUpType::BOUNCER) {  
                p.Velocity.y = fabsf(p.Velocity.y) * 0.85f + 260.0f;  
                p.Velocity.x = p.Velocity.x * 1.15f;  
                cout << "[POWERUP] Bouncer Hit! Launched Skyward!\n";  
            }  
            else if (pu.type == PowerUpType::BOOST_RING) {  
                glm::vec3 dir = glm::normalize(p.Velocity);  
                p.Velocity += dir * 220.0f;  
                cout << "[POWERUP] Boost Ring Passed! Accelerated Flight!\n";  
            }  
        }  
    }  
}
```

5. Win / Progression & Water Failure Conditions

- **Mechanic:** Collect enough bottles during flight to advance levels; landing/falling below the water line or out-of-bounds causes turn failure.

```
auto result = pirategame::checkCollisions(pirate, gBottles, gLevel.waterLine);
```

```
// Win condition
int currentHitCount = pirategame::bottlesCollected(gBottles);
if (currentHitCount >= gLevel.bottleQuota) {
    onFlightEnd(false);
    break;
}
```

```
// Lose/Fail Condition
bool outOfBounds = pirate.Position.x > ORTHO_SIZE + 50.0f
    || pirate.Position.x < -ORTHO_SIZE - 50.0f;

if (result.hitWater || outOfBounds) {
    onFlightEnd(result.hitWater);
    break;
}
```